PA07_Theory

1.
Pass-by-value:
	The function receives copies of the arguments. The activation record starts with a = 1 and b = 2. The first statement sets t = 1. Next, a = b changes a to 2, and b = t changes b to 1. The callee finishes with a = 2, b = 1, and t = 1. Since the parameters are copies, the caller's values do not change. The final caller values are x = 1 and y = 2.

Pass-by-reference:
	The parameters refer directly to the variables. Initially, a refers to x = 1, and b refers to y = 2. The first statement sets t = 1. Then a = b changes x to 2, and b = t changes y to 1. The callee finishes with a = 2, b = 1, and t = 1. Because the parameters share the caller's storage, the final caller values are x = 2 and y = 1.

Pass-by-value-result:
	The function starts with local copies of the arguments, so the activation record initially has a = 1 and b = 2. The first statement sets t = 1, then a = b changes a to 2, and b = t changes b to 1. At the end of the function, the callee has a = 2, b = 1, and t = 1. When the function returns, a is copied back to x, and b is copied back to y. The final caller values are x = 2 and y = 1.

2.
Pass-by-reference:
	Both parameters refer to the same caller variable, a. Initially, x = 10 and y = 10. When x = x + 1 executes, the shared variable becomes 11. Then y = y + 1 increments that same variable to 12. The final caller value is a = 12.

Pass-by-value-result:
	The function receives two separate local copies, so x = 10 and y = 10, while the caller's a remains 10. The first assignment changes local x to 11, and the second assignment changes local y to 11. When the function returns, the values are copied back in parameter order. First, x = 11 is copied to a, making a = 11. Then y = 11 is copied to a, leaving a = 11. The final caller value is a = 11.

The results are different because pass-by-reference updates the same variable twice, while pass-by-value-result updates two local copies, then copies them back in order. The second copy overwrites the first, which is why the final values are 12 for pass-by-reference and 11 for pass-by-value-result.

3. All three closures return 3 when called after the loop finishes. This happens because each closure captures the environment containing i, not just a snapshot of its value. All three closures refer to the same variable in the same environment. Once the loop finishes, i = 3, so each closure returns 3.

This relates to the static link concept from PA 6 because a static link provides access to the environment associated with a function's scope. Closures retain access to their defining environment, meaning they can resolve variables even when called

from somewhere else. Since all three closures capture the same environment, they all return the same answer.

To make each closure return a different value, the for loop could be changed so that each iteration creates a new binding of i. Each closure would then capture the environment containing that iteration's own value. This would produce return values of 0, 1, and 2, similar to JavaScript's let-per-iteration behavior. The closure mechanism itself would not need to change, though, only the loop's behavior.